

What are MDS Matrices?

Alexander Michos | September 6, 2026

I love it when two seemingly unrelated ideas in math collide together in a beautiful way: in this note I want to introduce what an MDS matrix is from the context of Error Correcting Codes and motivate why they appear in an astounding number of real-world encryption algorithms: AES, Twofish, LED, Square, etc. not to mention cryptographic hashes like Whirlpool and Photon.

A **Maximum Distance Separable Matrix** is a linear map $M : \mathbb{F}^a \rightarrow \mathbb{F}^b$ having the property that $\min_{0 \neq x \in \mathbb{F}^a} [w(x) + w(Mx)] = b + 1$ where w is the Hamming weight: the number of non-zero field elements in a given vector.

The term originates in the theory of linear codes: where you start with a generator matrix $G \in \mathbb{F}^{n \times k}$ and encode a message $x \in \mathbb{F}^k$ as simply Gx , and the space of all recognizable code-words is then $\text{img}(G)$. During transmission, various symbols (elements of $\mathbb{F}$) might get corrupted / flipped / substituted in that n -length vector, and so to ensure readability G should be chosen to maximize the **minimum distance between codewords**: $d = \min_{x \neq y \in \mathbb{F}^k} w(Gx - Gy) = \min_{z \neq 0 \in \mathbb{F}^k} w(Gz)$.

- For example, $G = [1, 1, 1]^\top$ is what most people would do to overcome a dodgy telephone line: take your message $x \in \mathbb{F}$ and repeat it 3 times $[x, x, x]^\top$ so that even if one of the symbols gets corrupted $[x, x', x]^\top$, a majority vote allows you to recover x . It's pretty clear that $d = 3$ since for any non-zero input message x , $[x, x, x]^\top$ has Hamming weight 3.

Linear codes are nicely described / characterized by the following data:

LINEAR CODE DISTANCE

Codes are labeled by $[n, k, d]$: n is the output length, k is the input length, and d is the minimum distance between codewords. Let G be the $n \times k$ generator matrix for such a linear code: it has minimum distance d if and only if **every collection** of $n - d + 1$ rows (not necessarily consecutive) have full column rank k .

- Suppose there was a collection of $n - d + 1$ rows which did not have full column rank: then there exists a non-zero nullspace vector x such that Gx has 0 entries in those $n - d + 1$ row positions. Clearly the vector Gx then has $\leq d - 1$ **non-zero** entries left which contradicts d being the minimal weight of a codeword. Conversely, any such vector x with $w(Gx) < d$ clearly gives rise to a set of $\leq n - d + 1$ rows which (when considered as a submatrix) has a nullspace vector x , contradicting any such collection having full column rank.

Clearly d is very important: the larger the distance between codewords, the more corruptions / errors must occur before one codeword gets so scrambled as to be interpreted as another one. But obviously d cannot be made arbitrarily large either: a finite amount room ($|\mathbb{F}|^n$ namely) must fit not only each of the code words, but also fit the d -meter social-distancing protocol between them.

SINGLETON BOUND

In any code (linear or otherwise) made up of n -tuples of q -symbols with minimum distance d there can be no more than q^{n-d+1} codewords in the code space. A code which attains this bound with equality is called **Maximum Distance Separable**. In particular, if the code is linear, then there are at most q^k codewords so $k \leq n - d + 1$ or $d \leq n - k + 1$.

- It's a sneaky pigeon-hole principle argument: since all the codewords differ from one another in d places, I can remove $d - 1$ symbols (say from the front of the list) from each and each codeword (now $n - [d - 1]$ in length) will still differ in ≥ 1 location. There are at most $q^{n-[d-1]}$ such codewords.

This is starting to look more like our initial definition! If we let $G = \begin{bmatrix} I \\ M \end{bmatrix}$ which has $n = a + b$ and $k = a$ and we'll notice that $w(Gz)$ is the sum of the blocks $w(Iz)$ and $w(Mz)$, meaning $d = \min_{0 \neq x \in \mathbb{F}^a} [w(x) + w(Mx)]$ which must be less than $[a + b] - [a] + 1 = b + 1$ and so an MDS matrix is the bottom portion of a linear code which attains the **maximum possible spacing** possible between codewords.

- That M is relegated to the “bottom portion” is more natural than you might expect: it corresponds to taking your message $x \in \mathbb{F}^k$ and *appending* Mx to the end where Mx are the “parity check bits” which tell you how and where to edit the corrupted message.

There's a nice way of thinking about checking whether every subset of $n - d + 1 = [a + b] - [b + 1] + 1 = a$ rows of G have full rank: recall this is an equivalent condition to G generating a code of distance $d = b + 1$ which is what is required to meet the singleton bound. That nice strategy is to check whether every **possible sub matrix** of M is invertible:

- Suppose you grab r rows of I and $a - r$ rows of M in your a -subcollection from G . WLOG permute the columns of this sub-matrix so that an $r \times r$ identity matrix is formed in the upper left, leaving an $a - r \times a - r$ **sub-matrix** of M in the lower right. Clearly we have r pivots due to the $r \times r$ identity block, so the entire collection will be linearly independent if and only if the $a - r \times a - r$ sub-matrix has $a - r$ pivots in it too: which is to say that it is non-singular. By varying r and the choice of rows, it's not hard to see that every possible sub-matrix of M is scrutinized in this way.

Exercise: Show that no MDS matrix contains a 0 and that every square MDS matrix is invertible. Deduce that over $\mathbb{F}_2$ there are no MDS codes. Suppose $\alpha \in \mathbb{F}_{q^k}$ lives strictly outside of the ground field $\mathbb{F}_q$. Show that $\begin{bmatrix} \alpha & 1 & 1 \\ 1 & \alpha & 1 \\ 1 & 1 & \alpha \end{bmatrix}$ is MDS. Is the same true for a 4×4 example of this type?

Cryptographic Intuition

Now for the connection to cryptography: if you were an attacker looking to stage a differential attack on a cipher, you are intuitively looking for “small changes” Δx to pass through the cipher and generate “small changes” Δy on the other end: the fewer bits that get affected the better. In the worst case, you would find that no matter what small change Δx you make, a huge number of bits of the output change: obfuscating your analysis behind a mountain of the **avalanche effect**.

- However, by employing a 4×4 MDS matrix M for example in the bowels of the algorithm, Mx and the differential $M(x + \Delta x)$ are **guaranteed** to differ in $\geq r$ locations where $w(\Delta x) = 5 - r$: this is a direct result of the fact that the distance between Mx and $M(x + \Delta x)$ is $w(M\Delta x)$ which can be no smaller than $b + 1 - w(\Delta x)$. In particular, changing 1 symbol of x would put $w(\Delta x) = 1$ and hence **all** 4 of the outputs would shift by a non-zero amount.
- Here's a similar bit of intuition: recall that an MDS matrix cannot contain a 0. The row which contained a 0 would be immune to any changes in the variable whose column contained that 0 and therefore no diffusion of that input column to that output row would occur.

So delightfully, what started as a method of **separating codewords** for distinguishability works perfectly to **efficiently diffuse** small changes in the input to many bits of the output, one of the foundational principles of cryptographic primitives like ciphers and hashes!

On the one hand, $\min_{0 \neq x \in \mathbb{F}^a} [w(x) + w(Mx)] = b + 1$ is telling us that code-words are separated by high Hamming weight, and on the other hand $\min_{0 \neq x \in \mathbb{F}^a} [w(x) + w(Mx)] = b + 1$ is telling us that even subtle, low-Hamming-weight perturbations to the plaintext must be accompanied by correspondingly large perturbations to the output Mx . It's a simple duality to write down, but I think it's pretty genius.

Let's test our understanding by pulling an MDS out of a real-world encryption algorithm: Twofish.

```
R.<x> = GF(2)[]
modulus = x^8 + x^6 + x^5 + x^3 + 1
F.<a> = GF(2^8, modulus=modulus) #Underlying field of Twofish
to_field = lambda byte_val: F(Integer(byte_val).bits())

M=[
    [1, 239, 91, 91],
    [91, 239, 239, 1],
    [239, 91, 1, 239],
    [239, 1, 239, 91],
```

```

] #The MDS matrix in question
M=matrix(F,[[to_field(x) for x in row] for row in M])

G=identity_matrix(F,4).stack(M)
import itertools
import random
for row_idx in itertools.combinations(range(8),4):
    submatrix=G[row_idx,:]
    print("Full rank?",submatrix.rank()==4) #Are the rows of the submatrix of G linearly independent?

for _ in range(30):
    x=random_vector(F,4)
    y=M*x #Output
    x[random.randint(0,3)]+=F.random_element() #Corrupting one bit of the input
    delta_y=y-M*x
    print("Number of Changes",sum([dy!=0 for dy in delta_y]))

```

Or what about AES? Which uses another 4×4 MDS over a slightly different presentation of the field $\mathbb{F}_{2^8}$:

```

R.<x> = GF(2)[x]
modulus = x^8 + x^4 + x^3 + x + 1 #Rijndael's finite field
F.<a> = GF(2^8, modulus=modulus)
to_field = lambda byte_val: F(Integer(byte_val).bits())

M=[
    [2, 3, 1, 1],
    [1, 2, 3, 1],
    [1, 1, 2, 3],
    [3, 1, 1, 2],
] #The MDS matrix in question

M=matrix(F,[[to_field(x) for x in row] for row in M])

import itertools
def is_MDS(mat):
    n=mat.dimensions()[0]
    for sub_matrix_size in range(1,n+1):
        for row_idx in itertools.combinations(range(n),sub_matrix_size):
            for col_idx in itertools.combinations(range(n),sub_matrix_size):
                if not mat[row_idx,col_idx].is_invertible():
                    return False
    return True

print(is_MDS(M))

```

Not only do Error Correcting Codes give us the notion of MDS matrices, but practical code designs with serious engineering applications can be used directly in cipher design: For example, the SHARK cipher contains an MDS matrix directly derived from the Reed Solomon Code, the very same coding technology by which DVDs / CDs / Blu-Rays were made scratch-resistant! It should come as no surprise that Reed Solomon Codes are of type $[n, k, n - k + 1]$ and hence maximum distance separable.

An Inverse Treat

To state the obvious, encryption and decryption under a symmetric cipher must be inverses of each other, and we would hope that if $S_1, S_2, \dots, S_N$ are the sequence of “secure steps” used to encrypt the plaintext then the decryption phase $S_N^{-1}, \dots, S_2^{-1}, S_1^{-1}$ made up of the inverse “steps” in the reverse order are equally secure. So, it’s natural to ask if M^{-1} is as “secure” as M .

Beautifully, if M is MDS then it’s inverse (recall MDS implies invertible) will also be MDS: meaning decryption **naturally inherits the differential resistance** that we planned for encryption! The proof is a one-liner given this lemma:

JACOBI COMPLEMENT MINOR FORMULA

Let A be any square matrix and $I, J \subseteq \{1, \dots, n\}$ be collections of row and column indices of the same size k , so $A[I, J]$ is a $k \times k$ submatrix of A . Then,

$$\det(A[I, J]) = \pm \det(A) \det(A^{-1}[J^c, I^c])$$

- As a result, if every $k \times k$ submatrix of M has full rank (M is MDS), then every $n - k \times n - k$ submatrix of M^{-1} will have non-vanishing determinant, making it also MDS.
- (Proof from MathSE) Start in the simple case where $I = J = \{1, \dots, k\}$ and so the set of columns $A[:, \{1, \dots, k\}]$ can be thought of as the action of A on the standard basis vectors $e_1, \dots, e_k$ while the columns $A^{-1}[:, \{k+1, \dots, n\}]$ can be thought of as the pre-images of the remaining basis vectors $e_{k+1}, \dots, e_n$ under A . This allows us to build a sneaky block matrix where $A^{-1}[:, \{k+1, \dots, n\}]$ and $A[:, \{1, \dots, k\}]$ flip places:

$$A \begin{bmatrix} e_1, \dots, e_k & A^{-1}[:, \{k+1, \dots, n\}] \end{bmatrix} = \begin{bmatrix} A[:, 1], \dots, A[:, k] & e_{k+1}, \dots, e_n \end{bmatrix}$$

$$A \begin{bmatrix} I_k & A^{-1}[\{1, \dots, k\}, \{k+1, \dots, n\}] \\ 0 & A^{-1}[\{k+1, \dots, n\}, \{k+1, \dots, n\}] \end{bmatrix} = \begin{bmatrix} A[\{1, \dots, k\}, \{1, \dots, k\}] & 0 \\ A[\{k+1, \dots, n\}, \{1, \dots, k\}] & I_{n-k} \end{bmatrix}$$

- And then since the determinant of a triangular block matrix is the determinant of the diagonal blocks we get something which looks like a “complement submatrix” of A^{-1} :

$$\implies \det(A) \det(I_k A^{-1}[\{k+1, \dots, n\}, \{k+1, \dots, n\}]) = \det(A[\{1, \dots, k\}, \{1, \dots, k\}] I_{n-k})$$

- Obviously $I = J = \{1, \dots, k\}$ requires some WLOG’ing: by shuffling around the rows of A we can put the I -rows in the first k positions and similarly by shuffling the columns we can put the J -columns in the first k columns, which looks like multiplying by two permutation matrices P, P' such that PAP' has $A[I, J]$ in the top left $k \times k$ position. This contributes the $\pm$ to $\det(A)$ since row/column swaps transpositions each pick up a negative sign.
- Note that $[PAP']^{-1} = P'^{-1}A^{-1}P^{-1}$ swaps the effect of P and P' so now the row permutations and column permutations applied to A now serve as column permutations and row permutations applied to A^{-1} : in particular $P'^{-1}A^{-1}P^{-1}[\{k+1, \dots, n\}, \{k+1, \dots, n\}]$ gets carried back to $P'^{-1}A^{-1}[\{k+1, \dots, n\}, I^c]$ which gets carried back to $A^{-1}[J^c, I^c]$. This is why we see J^c, I^c in a swapped order with respect to $A[I, J]$. Hopefully it makes sense why P^{-1} is the permutation sending $\{1, \dots, k\}$ to I and $\{k+1, \dots, n\}$ to I^c : it’s a bijection undoing the shuffling which carried I into the first k rows.

Circulant Counting Puzzle

Circulant matrices, ie those which look like evenly shifted rows $A = \begin{bmatrix} c_0 & c_1 & c_2 & \dots & c_{n-1} \\ c_{n-1} & c_0 & c_1 & \dots & c_{n-2} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ c_1 & c_2 & c_3 & \dots & c_0 \end{bmatrix}$, are an extremely popular choice for MDS-based ciphers since there are only n **scalar multiplications** needed to be programmed in software/hardware instead of $\sim n^2$: all of the rows consist of those same n field operations applied in some different order to the incoming input.

Obviously, circulant matrices aren’t the only kind which provide this cost-saving property: we saw AES went for a circulant $[2, 3, 1, 1]$ construction while TwoFish went for a more random pattern.

Since encryption algorithms are implemented on computers (to state the obvious), MDS matrices which operate over binary fields $\mathbb{F}_{2^k}$ are by far the most important category studied today. My parting exercise to you is to count these objects for small n :

Exercise: Show that the number of circulant MDS matrices over $\mathbb{F}_q$ is equal to $q - 1$ times the number of circulant MDS matrices over $\mathbb{F}_q$ with $c_0 = 1$: this is the *scale-invariance* property of MDS matrices. Recalling that $|\mathbb{F}_{2^k}^*| = 2^k - 1$ is not even, show that there are no non-trivial solutions to $\alpha^2 = 1$ in $\mathbb{F}_{2^k}$. Deduce that for $n = 1, 2$ there are $\prod_{j=0}^{n-1} (2^k - 2^j)$ circulant MDS matrices over $\mathbb{F}_{2^k}$. Using the `is_MDS` function from above, compute the count for $n = 3$ and $k = 3$. Do these formulas match? What about $n = 3$ and $k = 4$?

Interestingly, while the $\prod_{j=0}^{n-1} (2^k - 2^j)$ formula definitely fails in general it can cling on in the $n = 3, k$ -odd regime: the proof is very tedious/boring casework on the variables α, β making up the circulant row $[1, \alpha, \beta]$: you conclude $\beta \notin \{\alpha^{-1}, \alpha^2, \sqrt{\alpha}, \alpha + 1\}$ which is why you have $2^k - 4$ options, multiplied by the $2^k - 2$ choices for $\alpha \notin \{0, 1\}$, multiplied by the $2^k - 1$ WLOG factor. The oddness of k comes down to the fact that $\alpha^3 = 1$ should have no non-trivial solutions in $\mathbb{F}_{2^k}$ since $3 \nmid |\mathbb{F}_{2^k}^*| = 2^k - 1$.